

High-Concurrency Booking Engine (BookMyShow-style)

Database design for a movie ticketing backend where thousands of users compete for the same seats. It covers:

- **P1:** entities, attributes, tables, normalization (1NF to BCNF), sample rows, and a locking strategy that prevents double booking and lost holds.
- **P2:** a query listing every show at a theatre on a given date, with its show timings.

All SQL targets **MySQL 8.0.16+** (InnoDB). It uses recursive CTEs, `JSON_TABLE` and enforced `CHECK` constraints.

File	Purpose
sql/01_schema.sql	P1: <code>CREATE TABLE</code> statements
sql/02_sample_data.sql	P1: sample rows (shows generated for the next 7 days from <code>CURDATE()</code>)
sql/03_p2_shows_by_theatre_and_date.sql	P2: query, plus date-strip query and <code>EXPLAIN</code>
sql/04_booking_flow.sql	Stored procedures for seat hold, idempotent webhook and hold expiry, with a demo
tests/race_same_seats.sh	Concurrency test: N sessions race for the same seats
tests/load_test.py	Load test: 2,000 virtual users with holds, retries, payments, duplicate webhooks and expiry, followed by invariant checks

The full P1 SQL (schema and sample data) is also reproduced in this document under [P1: SQL](#), and the P2 SQL under [P2](#). The complete P2 script and the stored procedures are in [Appendix A](#) and [Appendix B](#).

How to run

```
mysql -uroot -p < sql/01_schema.sql
mysql -uroot -p < sql/02_sample_data.sql
mysql -uroot -p --table < sql/03_p2_shows_by_theatre_and_date.sql
mysql -uroot -p --table < sql/04_booking_flow.sql
```

Or with Docker:

```
docker run -d --name bms-mysql -e MYSQL_ROOT_PASSWORD=root -v "$PWD/sql:/sql" mysql:8.0
docker exec bms-mysql sh -c "mysql -uroot -proot < /sql/01_schema.sql"
# ...same for 02, 03, 04
docker cp tests/race_same_seats.sh bms-mysql:/tmp/race.sh
docker exec bms-mysql sh /tmp/race.sh 50

# Load test in a throwaway Python container that shares only bms-mysql's network
docker run --rm --network container:bms-mysql -v "$PWD/tests:/tests" python:3.12-slim \
```

```
sh -c "pip install -q mysql-connector-python && python /tests/load_test.py --host 127.0.0.1 --users 2000 --workers 100"
```

P1: Entities and relationships

```
city 1-* theatre 1-* screen 1-* seat *-1 seat_category
      |
      1
      *
movie 1-* movie_show *-1 language
      |
      *   1-* show_format
      *   1-* show_price *-1
movie_genre 1-* show_seat *-1 seat
      *
      1-* booking 1-* booking_seat
genre
      |   *-1 app_user
      1-* payment 1-* payment_webhook_event
```

- A **theatre** belongs to a **city** and has many **screens**.
- A **screen** has a fixed layout of **seats**, and each seat has a **seat_category** (Silver, Gold, Recliner).
- A **movie_show** is one screening of a movie on a screen at a start time, in one language and one format (2D, 3D, IMAX). (**show** is a reserved word in MySQL, hence the name.)
- **show_price** gives the price of each seat category for a show.
- **show_seat** is the per-show seat inventory: one row per (show, seat). This is the row that gets locked.
- A **booking** starts as a timed hold (**PENDING**) and becomes **CONFIRMED** once payment succeeds. **booking_seat** records which seats it covered and the price charged for each.
- **payment** tracks the gateway order. **payment_webhook_event** stores each webhook delivery exactly once.

P1: Tables, attributes and example rows

PK = primary key, FK = foreign key, UK = unique key.

city

Column	Type	Constraints
city_id	INT UNSIGNED	PK, auto-increment
name	VARCHAR(100)	NOT NULL, UK(name, state)
state	VARCHAR(100)	NOT NULL

city_id	name	state
1	Bengaluru	Karnataka
2	Mumbai	Maharashtra

theatre

Column	Type	Constraints
theatre_id	INT UNSIGNED	PK
city_id	INT UNSIGNED	FK → city
name	VARCHAR(150)	UK(city_id, name)
address	VARCHAR(255)	NOT NULL
pincode	CHAR(6)	NOT NULL

theatre_id	city_id	name	address	pincode
1	1	PVR Orion Mall	Dr Rajkumar Rd, Rajajinagar	560055
2	1	INOX Garuda Mall	Magrath Rd, Ashok Nagar	560025
3	2	Cinepolis Andheri West	Fun Republic Mall, Andheri W	400053

screen

Column	Type	Constraints
screen_id	INT UNSIGNED	PK
theatre_id	INT UNSIGNED	FK → theatre, UK(theatre_id, name)
name	VARCHAR(50)	NOT NULL

screen_id	theatre_id	name
1	1	Audi 1
2	1	Audi 2
3	2	Screen 1

seat_category

Column	Type	Constraints
category_id	TINYINT UNSIGNED	PK
name	VARCHAR(30)	UK

category_id	name
1	SILVER
2	GOLD
3	RECLINER

seat (physical layout, independent of shows)

Column	Type	Constraints
seat_id	INT UNSIGNED	PK

Column	Type	Constraints
screen_id	INT UNSIGNED	FK → screen
row_label	VARCHAR(3)	UK(screen_id, row_label, seat_number)
seat_number	SMALLINT UNSIGNED	
category_id	TINYINT UNSIGNED	FK → seat_category

seat_id	screen_id	row_label	seat_number	category_id
1	1	A	1	1
25	1	C	5	2
41	1	E	1	3

movie

Column	Type	Constraints
movie_id	INT UNSIGNED	PK
title	VARCHAR(200)	UK(title, release_date)
duration_min	SMALLINT UNSIGNED	CHECK > 0
certification	ENUM('U','UA','UA7+','UA13+','UA16+','A','S')	
release_date	DATE	

movie_id	title	duration_min	certification	release_date
1	Kantara: Chapter 1	168	UA13+	2025-10-02
2	Dune: Part Two	166	UA	2024-03-01
3	Inside Out 2	96	U	2024-06-14

genre and movie_genre (many-to-many)

genre_id	name
1	Action
5	Thriller

movie_id	genre_id
1	1
1	5

movie_genre PK is (movie_id, genre_id). Both columns are FKs.

language and show_format

language_id	name
1	Kannada

language_id	name
3	English

format_id	name
1	2D
3	IMAX 2D

movie_show

Column	Type	Constraints
show_id	BIGINT UNSIGNED	PK
screen_id	INT UNSIGNED	FK → screen, UK(screen_id, start_time)
movie_id	INT UNSIGNED	FK → movie, KEY(movie_id, start_time)
language_id	SMALLINT UNSIGNED	FK → language
format_id	TINYINT UNSIGNED	FK → show_format
start_time	DATETIME	theatre-local time
end_time	DATETIME	CHECK end_time > start_time
status	ENUM('SCHEDULED','CANCELLED')	default SCHEDULED

show_id	screen_id	movie_id	language_id	format_id	start_time	end_time	status
7	1	1	1	1	2026-09-30 09:30:00	2026-09-30 12:45:00	SCHEDULED
14	1	1	1	1	2026-09-30 13:15:00	2026-09-30 16:30:00	SCHEDULED
21	1	2	3	3	2026-09-30 17:00:00	2026-09-30 20:15:00	SCHEDULED

show_price

Column	Type	Constraints
show_id	BIGINT UNSIGNED	PK part, FK → movie_show
category_id	TINYINT UNSIGNED	PK part, FK → seat_category
price	DECIMAL(10,2)	CHECK ≥ 0

show_id	category_id	price
7	1	180.00
7	2	250.00
21	3	650.00

app_user

Column	Type	Constraints
user_id	BIGINT UNSIGNED	PK
name	VARCHAR(100)	
email	VARCHAR(255)	UK
phone	VARCHAR(15)	UK
created_at	DATETIME	

user_id	name	email	phone
1	Aarav Sharma	aarav@example.com	9876500001
2	Diya Patel	diya@example.com	9876500002

booking

Column	Type	Constraints
booking_id	BIGINT UNSIGNED	PK
user_id	BIGINT UNSIGNED	FK → app_user
show_id	BIGINT UNSIGNED	FK → movie_show
status	ENUM('PENDING','CONFIRMED','EXPIRED','CANCELLED','FAILED')	
hold_expires_at	DATETIME	KEY(status, hold_expires_at)
idempotency_key	CHAR(36)	UK(user_id, idempotency_key)
created_at, updated_at	DATETIME	

booking_id	user_id	show_id	status	hold_expires_at	idempotency_key
1	1	7	CONFIRMED	2026-09-30 10:47:27	6f1c2b7e-...
2	2	7	PENDING	2026-09-30 10:47:27	a9d8e7f6-...
3	3	7	EXPIRED	2026-09-30 10:32:27	c0ffee00-...

show_seat (the inventory row that gets locked)

Column	Type	Constraints
show_id	BIGINT UNSIGNED	PK(show_id, seat_id) , FK → movie_show
seat_id	INT UNSIGNED	FK → seat
booking_id	BIGINT UNSIGNED NULL	FK → booking. Current owner, NULL if free
version	INT UNSIGNED	optimistic-lock counter

show_id	seat_id	booking_id	version
7	1	NULL	2
7	35	1	1
7	41	2	1

booking_seat

Column	Type	Constraints
booking_id	BIGINT UNSIGNED	PK part, FK → booking
seat_id	INT UNSIGNED	PK part, FK → seat
price_paid	DECIMAL(10,2)	price snapshot at booking time

booking_id	seat_id	price_paid
1	35	250.00
1	36	250.00
2	41	450.00

payment

Column	Type	Constraints
payment_id	BIGINT UNSIGNED	PK
booking_id	BIGINT UNSIGNED	FK → booking
gateway	VARCHAR(30)	UK(gateway, gateway_order_id)
gateway_order_id	VARCHAR(100)	
amount	DECIMAL(10,2)	CHECK ≥ 0
status	ENUM('INITIATED','SUCCESS','FAILED','REFUND_PENDING','REFUNDED')	

payment_id	booking_id	gateway	gateway_order_id	amount	status
1	1	RAZORPAY	order_Q1a2b3c4d5	500.00	SUCCESS
2	2	RAZORPAY	order_Q9z8y7x6w5	900.00	INITIATED

payment_webhook_event

Column	Type	Constraints
event_id	BIGINT UNSIGNED	PK
gateway	VARCHAR(30)	UK(gateway, gateway_event_id)
gateway_event_id	VARCHAR(100)	
payment_id	BIGINT UNSIGNED	FK → payment
event_type	VARCHAR(50)	

Column	Type	Constraints
payload	JSON	
received_at, processed_at	DATETIME	

event_id	gateway	gateway_event_id	payment_id	event_type	processed_at
1	RAZORPAY	evt_Q1a2b3c4d5e6	1	payment.captured	2026-09-30 10:37:27

P1: SQL

This is identical to `sql/01_schema.sql` and `sql/02_sample_data.sql` . Run the schema first, then the sample data.

Schema (CREATE TABLE)

```
-- =====
-- High-Concurrency Booking Engine (BookMyShow-style) - P1 Schema
-- Target: MySQL 8.0.16+ (InnoDB, utf8mb4, CHECK constraints enforced)
-- =====

DROP DATABASE IF EXISTS booking_engine;
CREATE DATABASE booking_engine CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE booking_engine;

-- -----
-- Location & venue
-- -----

CREATE TABLE city (
    city_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(100) NOT NULL,
    state        VARCHAR(100) NOT NULL,
    PRIMARY KEY (city_id),
    UNIQUE KEY uq_city_name_state (name, state)
) ENGINE = InnoDB;

CREATE TABLE theatre (
    theatre_id   INT UNSIGNED NOT NULL AUTO_INCREMENT,
    city_id      INT UNSIGNED NOT NULL,
    name         VARCHAR(150) NOT NULL,
    address      VARCHAR(255) NOT NULL,
    pincode      CHAR(6)      NOT NULL,
    PRIMARY KEY (theatre_id),
    UNIQUE KEY uq_theatre_city_name (city_id, name),
    CONSTRAINT fk_theatre_city FOREIGN KEY (city_id) REFERENCES city (city_id)
) ENGINE = InnoDB;
```

```

CREATE TABLE screen (
    screen_id    INT UNSIGNED    NOT NULL AUTO_INCREMENT,
    theatre_id   INT UNSIGNED    NOT NULL,
    name         VARCHAR(50)     NOT NULL,
    PRIMARY KEY (screen_id),
    UNIQUE KEY uq_screen_theatre_name (theatre_id, name),
    CONSTRAINT fk_screen_theatre FOREIGN KEY (theatre_id) REFERENCES theatre (theatre_id)
) ENGINE = InnoDB;

CREATE TABLE seat_category (
    category_id  TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(30)      NOT NULL,
    PRIMARY KEY (category_id),
    UNIQUE KEY uq_seat_category_name (name)
) ENGINE = InnoDB;

-- Physical seat layout of a screen (independent of any show).
CREATE TABLE seat (
    seat_id      INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    screen_id     INT UNSIGNED      NOT NULL,
    row_label    VARCHAR(3)        NOT NULL,
    seat_number   SMALLINT UNSIGNED NOT NULL,
    category_id  TINYINT UNSIGNED  NOT NULL,
    PRIMARY KEY (seat_id),
    UNIQUE KEY uq_seat_position (screen_id, row_label, seat_number),
    CONSTRAINT fk_seat_screen   FOREIGN KEY (screen_id)   REFERENCES screen (screen_id),
    CONSTRAINT fk_seat_category FOREIGN KEY (category_id) REFERENCES seat_category (category_id)
) ENGINE = InnoDB;

-- -----
-- Catalogue
-- -----

CREATE TABLE movie (
    movie_id      INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    title         VARCHAR(200)      NOT NULL,
    duration_min  SMALLINT UNSIGNED NOT NULL,
    certification ENUM('U','UA','UA7+','UA13+','UA16+','A','S') NOT NULL,
    release_date  DATE              NOT NULL,
    PRIMARY KEY (movie_id),
    UNIQUE KEY uq_movie_title_release (title, release_date),
    CONSTRAINT chk_movie_duration CHECK (duration_min > 0)
) ENGINE = InnoDB;

CREATE TABLE genre (
    genre_id      SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name          VARCHAR(50)        NOT NULL,
    PRIMARY KEY (genre_id),

```

```

        UNIQUE KEY uq_genre_name (name)
    ) ENGINE = InnoDB;

-- Many-to-many: a movie has many genres (kept out of `movie` for 1NF).
CREATE TABLE movie_genre (
    movie_id    INT UNSIGNED          NOT NULL,
    genre_id    SMALLINT UNSIGNED NOT NULL,
    PRIMARY KEY (movie_id, genre_id),
    KEY idx_movie_genre_genre (genre_id),
    CONSTRAINT fk_mg_movie FOREIGN KEY (movie_id) REFERENCES movie (movie_id),
    CONSTRAINT fk_mg_genre FOREIGN KEY (genre_id) REFERENCES genre (genre_id)
) ENGINE = InnoDB;

CREATE TABLE language (
    language_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name        VARCHAR(50)      NOT NULL,
    PRIMARY KEY (language_id),
    UNIQUE KEY uq_language_name (name)
) ENGINE = InnoDB;

CREATE TABLE show_format (
    format_id    TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(30)      NOT NULL,    -- 2D, 3D, IMAX 2D, 4DX ...
    PRIMARY KEY (format_id),
    UNIQUE KEY uq_show_format_name (name)
) ENGINE = InnoDB;

-- -----
-- Shows & pricing
-- (`show` is a reserved word in MySQL, hence `movie_show`)
-- -----

CREATE TABLE movie_show (
    show_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    screen_id    INT UNSIGNED     NOT NULL,
    movie_id     INT UNSIGNED     NOT NULL,
    language_id  SMALLINT UNSIGNED NOT NULL,
    format_id    TINYINT UNSIGNED NOT NULL,
    start_time   DATETIME         NOT NULL,    -- theatre-local time
    end_time     DATETIME         NOT NULL,    -- scheduled slot end (movie + ads + interval + cleaning)
    status       ENUM('SCHEDULED','CANCELLED') NOT NULL DEFAULT 'SCHEDULED',
    PRIMARY KEY (show_id),
    -- One show per screen per start time; also the access path for the P2 query.
    UNIQUE KEY uq_show_screen_start (screen_id, start_time),
    KEY idx_show_movie_start (movie_id, start_time),
    CONSTRAINT fk_show_screen FOREIGN KEY (screen_id) REFERENCES screen (screen_id),
    CONSTRAINT fk_show_movie FOREIGN KEY (movie_id) REFERENCES movie (movie_id),
    CONSTRAINT fk_show_language FOREIGN KEY (language_id) REFERENCES language (language_id),
    CONSTRAINT fk_show_format FOREIGN KEY (format_id) REFERENCES show_format (format_id),

```

```

        CONSTRAINT chk_show_times CHECK (end_time > start_time)
    ) ENGINE = InnoDB;

-- Price of each seat category for a given show.
CREATE TABLE show_price (
    show_id      BIGINT UNSIGNED NOT NULL,
    category_id  TINYINT UNSIGNED NOT NULL,
    price        DECIMAL(10,2)    NOT NULL,
    PRIMARY KEY (show_id, category_id),
    CONSTRAINT fk_price_show      FOREIGN KEY (show_id)      REFERENCES movie_show (show_id),
    CONSTRAINT fk_price_category FOREIGN KEY (category_id) REFERENCES seat_category (category_id),
    CONSTRAINT chk_price_positive CHECK (price >= 0)
) ENGINE = InnoDB;

-- -----
-- Users, bookings, seat inventory
-- -----

CREATE TABLE app_user (
    user_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(100)     NOT NULL,
    email        VARCHAR(255)     NOT NULL,
    phone        VARCHAR(15)      NOT NULL,
    created_at   DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id),
    UNIQUE KEY uq_user_email (email),
    UNIQUE KEY uq_user_phone (phone)
) ENGINE = InnoDB;

-- A booking starts life as a PENDING hold with an expiry and becomes
-- CONFIRMED only when a successful payment webhook is processed.
CREATE TABLE booking (
    booking_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id       BIGINT UNSIGNED NOT NULL,
    show_id       BIGINT UNSIGNED NOT NULL,
    status        ENUM('PENDING','CONFIRMED','EXPIRED','CANCELLED','FAILED') NOT NULL DEFAULT 'PENDING',
    hold_expires_at DATETIME       NOT NULL,
    idempotency_key CHAR(36)       NOT NULL, -- client-generated UUID; retries return the same booking
    created_at    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (booking_id),
    UNIQUE KEY uq_booking_idempotency (user_id, idempotency_key),
    KEY idx_booking_show (show_id),
    KEY idx_booking_status_expiry (status, hold_expires_at), -- expiry sweeper
    CONSTRAINT fk_booking_user FOREIGN KEY (user_id) REFERENCES app_user (user_id),
    CONSTRAINT fk_booking_show FOREIGN KEY (show_id) REFERENCES movie_show (show_id)
) ENGINE = InnoDB;

-- Seat inventory: exactly one row per (show, seat), created when the show

```

```

-- is published. The PRIMARY KEY makes it structurally impossible for a seat
-- to belong to two bookings at once; `booking_id` is the current owner
-- (NULL = never taken or released). Whether that owner still has a valid
-- claim is decided by booking.status / booking.hold_expires_at.
CREATE TABLE show_seat (
    show_id      BIGINT UNSIGNED NOT NULL,
    seat_id      INT UNSIGNED     NOT NULL,
    booking_id   BIGINT UNSIGNED NULL,
    version      INT UNSIGNED     NOT NULL DEFAULT 0, -- optimistic-locking counter
    PRIMARY KEY (show_id, seat_id),
    KEY idx_show_seat_booking (booking_id),
    CONSTRAINT fk_ss_show    FOREIGN KEY (show_id)    REFERENCES movie_show (show_id),
    CONSTRAINT fk_ss_seat    FOREIGN KEY (seat_id)    REFERENCES seat (seat_id),
    CONSTRAINT fk_ss_booking FOREIGN KEY (booking_id) REFERENCES booking (booking_id)
) ENGINE = InnoDB;

-- Immutable record of which seats a booking covered and what was charged.
-- price_paid is a historical snapshot: show_price may change later.
CREATE TABLE booking_seat (
    booking_id   BIGINT UNSIGNED NOT NULL,
    seat_id      INT UNSIGNED     NOT NULL,
    price_paid   DECIMAL(10,2)   NOT NULL,
    PRIMARY KEY (booking_id, seat_id),
    KEY idx_booking_seat_seat (seat_id),
    CONSTRAINT fk_bs_booking FOREIGN KEY (booking_id) REFERENCES booking (booking_id),
    CONSTRAINT fk_bs_seat    FOREIGN KEY (seat_id)    REFERENCES seat (seat_id)
) ENGINE = InnoDB;

-- -----
-- Payments & idempotent webhooks
-- -----

CREATE TABLE payment (
    payment_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    booking_id    BIGINT UNSIGNED NOT NULL,
    gateway       VARCHAR(30)      NOT NULL, -- RAZORPAY, STRIPE ...
    gateway_order_id VARCHAR(100)  NOT NULL,
    amount        DECIMAL(10,2)   NOT NULL,
    status        ENUM('INITIATED','SUCCESS','FAILED','REFUND_PENDING','REFUNDED') NOT NULL DEFAULT
'INITIATED',
    created_at    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (payment_id),
    UNIQUE KEY uq_payment_gateway_order (gateway, gateway_order_id),
    KEY idx_payment_booking (booking_id),
    CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id) REFERENCES booking (booking_id),
    CONSTRAINT chk_payment_amount CHECK (amount >= 0)
) ENGINE = InnoDB;

```

```

-- Every webhook delivery is recorded once. Gateways retry deliveries, so
-- the UNIQUE (gateway, gateway_event_id) key is the idempotency guard:
-- a duplicate INSERT fails and the handler simply returns 200.
CREATE TABLE payment_webhook_event (
    event_id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    gateway           VARCHAR(30)      NOT NULL,
    gateway_event_id  VARCHAR(100)     NOT NULL,
    payment_id        BIGINT UNSIGNED NOT NULL,
    event_type        VARCHAR(50)      NOT NULL,    -- payment.captured, payment.failed ...
    payload           JSON             NOT NULL,
    received_at       DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_at      DATETIME         NULL,
    PRIMARY KEY (event_id),
    UNIQUE KEY uq_webhook_gateway_event (gateway, gateway_event_id),
    KEY idx_webhook_payment (payment_id),
    CONSTRAINT fk_webhook_payment FOREIGN KEY (payment_id) REFERENCES payment (payment_id)
) ENGINE = InnoDB;

```

Sample data (INSERT)

```

-- =====
-- Sample data. Shows are generated for the next 7 days relative to
-- CURDATE(), so the P2 query returns rows whenever this script is run.
-- =====

USE booking_engine;

INSERT INTO city (city_id, name, state) VALUES
    (1, 'Bengaluru', 'Karnataka'),
    (2, 'Mumbai',    'Maharashtra');

INSERT INTO theatre (theatre_id, city_id, name, address, pincode) VALUES
    (1, 1, 'PVR Orion Mall',      'Dr Rajkumar Rd, Rajajinagar',    '560055'),
    (2, 1, 'INOX Garuda Mall',    'Magrath Rd, Ashok Nagar',       '560025'),
    (3, 2, 'Cinepolis Andheri West', 'Fun Republic Mall, Andheri W',  '400053');

INSERT INTO screen (screen_id, theatre_id, name) VALUES
    (1, 1, 'Audi 1'),
    (2, 1, 'Audi 2'),
    (3, 2, 'Screen 1'),
    (4, 3, 'Screen 1');

INSERT INTO seat_category (category_id, name) VALUES
    (1, 'SILVER'),
    (2, 'GOLD'),
    (3, 'RECLINER');

-- Small layout per screen: rows A-E, 10 seats each.

```

```

-- A-B = SILVER, C-D = GOLD, E = RECLINER.
INSERT INTO seat (screen_id, row_label, seat_number, category_id)
WITH RECURSIVE n AS (SELECT 1 AS num UNION ALL SELECT num + 1 FROM n WHERE num < 10),
rows_ AS (
    SELECT 'A' AS row_label, 1 AS category_id UNION ALL
    SELECT 'B', 1 UNION ALL
    SELECT 'C', 2 UNION ALL
    SELECT 'D', 2 UNION ALL
    SELECT 'E', 3
)
SELECT s.screen_id, r.row_label, n.num, r.category_id
FROM screen s CROSS JOIN rows_ r CROSS JOIN n
ORDER BY s.screen_id, r.row_label, n.num;

INSERT INTO movie (movie_id, title, duration_min, certification, release_date) VALUES
    (1, 'Kantara: Chapter 1', 168, 'UA13+', '2025-10-02'),
    (2, 'Dune: Part Two', 166, 'UA', '2024-03-01'),
    (3, 'Inside Out 2', 96, 'U', '2024-06-14'),
    (4, 'Oppenheimer', 180, 'UA16+', '2023-07-21');

INSERT INTO genre (genre_id, name) VALUES
    (1, 'Action'), (2, 'Drama'), (3, 'Sci-Fi'), (4, 'Animation'), (5, 'Thriller'), (6, 'History');

INSERT INTO movie_genre (movie_id, genre_id) VALUES
    (1, 1), (1, 5),
    (2, 1), (2, 3),
    (3, 4),
    (4, 2), (4, 6);

INSERT INTO language (language_id, name) VALUES
    (1, 'Kannada'), (2, 'Hindi'), (3, 'English'), (4, 'Tamil');

INSERT INTO show_format (format_id, name) VALUES
    (1, '2D'), (2, '3D'), (3, 'IMAX 2D');

-- Daily schedule template, expanded over the next 7 days.
-- (screen, movie, language, format, start time, slot length in minutes)
INSERT INTO movie_show (screen_id, movie_id, language_id, format_id, start_time, end_time)
WITH RECURSIVE d AS (SELECT 0 AS day_offset UNION ALL SELECT day_offset + 1 FROM d WHERE day_offset < 6),
tmpl AS (
    SELECT 1 AS screen_id, 1 AS movie_id, 1 AS language_id, 1 AS format_id, '09:30:00' AS t, 195 AS slot
UNION ALL
    SELECT 1, 1, 1, 1, '13:15:00', 195 UNION ALL
    SELECT 1, 2, 3, 3, '17:00:00', 195 UNION ALL
    SELECT 1, 1, 1, 1, '21:00:00', 195 UNION ALL
    SELECT 2, 3, 3, 2, '10:00:00', 120 UNION ALL
    SELECT 2, 3, 3, 1, '12:30:00', 120 UNION ALL
    SELECT 2, 4, 3, 1, '15:00:00', 210 UNION ALL
    SELECT 2, 1, 2, 1, '19:00:00', 195 UNION ALL

```

```

SELECT 2, 1, 2, 1, '22:30:00', 195 UNION ALL
SELECT 3, 2, 3, 1, '11:00:00', 195 UNION ALL
SELECT 3, 1, 1, 1, '15:00:00', 195 UNION ALL
SELECT 4, 4, 2, 1, '12:00:00', 210 UNION ALL
SELECT 4, 2, 2, 1, '18:30:00', 195
)
SELECT t.screen_id, t.movie_id, t.language_id, t.format_id,
       TIMESTAMP(CURDATE() + INTERVAL d.day_offset DAY, t.t),
       TIMESTAMP(CURDATE() + INTERVAL d.day_offset DAY, t.t) + INTERVAL t.slot MINUTE
FROM d CROSS JOIN tmp1 t;

-- One cancelled show, to prove P2 filters it out.
UPDATE movie_show
SET status = 'CANCELLED'
WHERE screen_id = 2 AND start_time = TIMESTAMP(CURDATE(), '22:30:00');

INSERT INTO show_price (show_id, category_id, price)
SELECT ms.show_id, sc.category_id,
       CASE sc.name WHEN 'SILVER' THEN 180.00 WHEN 'GOLD' THEN 250.00 ELSE 450.00 END
       + CASE WHEN ms.format_id = 3 THEN 200.00 ELSE 0 END
FROM movie_show ms CROSS JOIN seat_category sc;

-- Seat inventory: one row per (show, seat) of that show's screen.
INSERT INTO show_seat (show_id, seat_id)
SELECT ms.show_id, s.seat_id
FROM movie_show ms
JOIN seat s ON s.screen_id = ms.screen_id;

INSERT INTO app_user (user_id, name, email, phone) VALUES
(1, 'Aarav Sharma', 'aarav@example.com', '9876500001'),
(2, 'Diya Patel', 'diya@example.com', '9876500002'),
(3, 'Rohan Iyer', 'rohan@example.com', '9876500003');

-- -----
-- A confirmed booking (user 1, today's 09:30 show at PVR Orion, D5 + D6)
-- -----

SET @show_id := (SELECT show_id FROM movie_show
                 WHERE screen_id = 1 AND start_time = TIMESTAMP(CURDATE(), '09:30:00'));

INSERT INTO booking (booking_id, user_id, show_id, status, hold_expires_at, idempotency_key)
VALUES (1, 1, @show_id, 'CONFIRMED', NOW() + INTERVAL 10 MINUTE, '6f1c2b7e-0d3a-4f55-9a61-2d7c1e9b0a01');

INSERT INTO booking_seat (booking_id, seat_id, price_paid)
SELECT 1, s.seat_id, sp.price
FROM seat s
JOIN show_price sp ON sp.show_id = @show_id AND sp.category_id = s.category_id
WHERE s.screen_id = 1 AND s.row_label = 'D' AND s.seat_number IN (5, 6);

UPDATE show_seat ss

```

```

JOIN booking_seat bs ON bs.seat_id = ss.seat_id AND bs.booking_id = 1
SET ss.booking_id = 1, ss.version = ss.version + 1
WHERE ss.show_id = @show_id;

INSERT INTO payment (payment_id, booking_id, gateway, gateway_order_id, amount, status)
VALUES (1, 1, 'RAZORPAY', 'order_Q1a2b3c4d5', 500.00, 'SUCCESS');

INSERT INTO payment_webhook_event (gateway, gateway_event_id, payment_id, event_type, payload,
processed_at)
VALUES ('RAZORPAY', 'evt_Q1a2b3c4d5e6', 1, 'payment.captured',
        JSON_OBJECT('order_id', 'order_Q1a2b3c4d5', 'amount', 50000, 'currency', 'INR'), NOW());

-- -----
-- A live hold (user 2, same show, E1 + E2, expires in 10 minutes)
-- -----

INSERT INTO booking (booking_id, user_id, show_id, status, hold_expires_at, idempotency_key)
VALUES (2, 2, @show_id, 'PENDING', NOW() + INTERVAL 10 MINUTE, 'a9d8e7f6-1b2c-4d3e-8f90-112233445566');

INSERT INTO booking_seat (booking_id, seat_id, price_paid)
SELECT 2, s.seat_id, sp.price
FROM seat s
JOIN show_price sp ON sp.show_id = @show_id AND sp.category_id = s.category_id
WHERE s.screen_id = 1 AND s.row_label = 'E' AND s.seat_number IN (1, 2);

UPDATE show_seat ss
JOIN booking_seat bs ON bs.seat_id = ss.seat_id AND bs.booking_id = 2
SET ss.booking_id = 2, ss.version = ss.version + 1
WHERE ss.show_id = @show_id;

INSERT INTO payment (payment_id, booking_id, gateway, gateway_order_id, amount, status)
VALUES (2, 2, 'RAZORPAY', 'order_Q9z8y7x6w5', 900.00, 'INITIATED');

-- -----
-- An abandoned hold (user 3, A1, expired 5 minutes ago) - the seat is
-- still pointed at booking 3 but is free to be taken by anyone.
-- -----

INSERT INTO booking (booking_id, user_id, show_id, status, hold_expires_at, idempotency_key)
VALUES (3, 3, @show_id, 'PENDING', NOW() - INTERVAL 5 MINUTE, 'c0ffee00-1234-4abc-9def-000000000003');

INSERT INTO booking_seat (booking_id, seat_id, price_paid)
SELECT 3, s.seat_id, sp.price
FROM seat s
JOIN show_price sp ON sp.show_id = @show_id AND sp.category_id = s.category_id
WHERE s.screen_id = 1 AND s.row_label = 'A' AND s.seat_number = 1;

UPDATE show_seat ss
JOIN booking_seat bs ON bs.seat_id = ss.seat_id AND bs.booking_id = 3

```

```
SET ss.booking_id = 3, ss.version = ss.version + 1
WHERE ss.show_id = @show_id;
```

P1: Normalization

1NF (atomic values, no repeating groups). Every column holds one value. A movie's genres live in `movie_genre`, not in a comma-separated column. A booking's seats live in `booking_seat`, not in a `seats` list. The webhook `payload` is JSON, but it is an opaque audit blob that is never queried relationally.

2NF (no partial dependency on a composite key). These tables have composite keys: -

`movie_genre(movie_id, genre_id)`: has no non-key columns. - `show_price(show_id, category_id) → price`: the price depends on both the show and the category. - `show_seat(show_id, seat_id) → booking_id, version`: ownership is per seat per show. - `booking_seat(booking_id, seat_id) → price_paid`: the price charged for that seat in that booking.

Seat attributes such as row and category stay in `seat`. Show attributes stay in `movie_show`. None are copied into the composite-key tables.

3NF (no transitive dependency). - `movie_show` stores `screen_id` only. The theatre and city are reached through `screen → theatre → city` and are not copied onto the show. - Language, format and category names are stored once, in lookup tables. - Seat state (`AVAILABLE`, `HELD`, `BOOKED`) is **not stored**. It is derived from `show_seat.booking_id` joined to `booking.status` and `booking.hold_expires_at`. Storing it would create the dependency `booking_id → status`, and the two copies could disagree. - The hold expiry is stored once, on `booking`, not on every seat row.

BCNF (every determinant is a candidate key). The only non-trivial functional dependencies are from each table's PK or from a declared UNIQUE key. For example, in `theatre`, (`city_id`, `name`) determines everything, and it is a candidate key. In `seat`, (`screen_id`, `row_label`, `seat_number`) is a candidate key. `payment` has two candidate keys: `payment_id` and (`gateway`, `gateway_order_id`). No non-key attribute determines another attribute.

Two decisions look redundant but are not: - `movie_show.end_time` is **not** derivable from `movie.duration_min`. It is the scheduled slot end and includes ads, the interval and cleaning time, all set by the theatre. -

`booking_seat.price_paid` is **not** derivable from `show_price`. It is a snapshot, and prices can change after booking (dynamic pricing, offers).

P1: Concurrency and locking strategy

Goals

1. **No double booking**: a seat for a show can never have two confirmed owners.
2. **No lost holds**: a live hold can never be taken by someone else, and an expired hold must become available even if a background job crashes.
3. **Idempotent payments**: repeated or out-of-order webhooks never confirm twice, charge twice, or corrupt state.

1. Structural guarantee: one row per (show, seat)

`show_seat` has `PRIMARY KEY (show_id, seat_id)` and a single `booking_id` column. Since a seat has only one row per show, it cannot point at two bookings, whatever the application does. The locking below only decides *who* gets to write that pointer.

2. Seat hold: pessimistic row lock plus conditional update (`sp_hold_seats`)

```
START TRANSACTION;
INSERT INTO booking (... , status='PENDING', hold_expires_at = NOW() + INTERVAL 10 MINUTE);

-- Lock the requested seats in seat_id order (deterministic order, so no deadlocks)
SELECT ... FROM show_seat WHERE show_id = ? AND seat_id IN (...) ORDER BY seat_id FOR UPDATE;

-- Claim only seats that are free right now
UPDATE show_seat ss LEFT JOIN booking b ON b.booking_id = ss.booking_id
SET ss.booking_id = :new_booking, ss.version = ss.version + 1
WHERE ss.show_id = ? AND ss.seat_id IN (...)
  AND (ss.booking_id IS NULL
      OR b.status IN ('EXPIRED', 'CANCELLED', 'FAILED')
      OR (b.status = 'PENDING' AND b.hold_expires_at <= NOW()));

-- All-or-nothing: if ROW_COUNT() <> number of seats requested, ROLLBACK
COMMIT;
```

- **Why pessimistic here?** For a hot show, many users go for the same seats at the same moment. With optimistic locking, most of them would read, compute, and then fail at write time and retry, which wastes work and hammers the database. `FOR UPDATE` on a handful of seat rows queues contenders for a few milliseconds each, and seats that don't overlap never block each other (InnoDB locks rows, not the table).
- **Where optimistic fits:** the `version` column supports `UPDATE ... WHERE version = :seen` for low-contention paths, such as an admin blocking seats or a seat-map cache refresh. It is also useful if the hold is moved into Redis and the database is written asynchronously.
- **No lost holds:** expiry is checked inside the claim query (`hold_expires_at <= NOW()`). An expired hold is therefore claimable even if the sweeper never runs, and a live hold is never claimable.
- **Client retries:** `UNIQUE (user_id, idempotency_key)` makes a retried "hold seats" request return the original booking instead of creating a second one.

3. Hold expiry (`sp_expire_holds` and `ev_expire_holds`)

A MySQL `EVENT` runs every 30 seconds. It marks expired `PENDING` bookings as `EXPIRED` and sets their `show_seat.booking_id` back to `NULL`, in batches. It picks candidates without locking them (`READ COMMITTED`), locks their seats, then re-checks each booking before expiring it, so a booking confirmed in the meantime is left alone. This job only tidies up; correctness comes from the claim query above. Holds with a payment still `INITIATED` get a 5-minute grace period before being swept.

Lock ordering (no deadlocks)

All three procedures take locks in the same order: `show_seat` rows in `seat_id` order first, then `booking` rows. The webhook handler locks the `payment` row before either of these, but nothing else ever locks `payment`. A consistent order means no two transactions can each hold a lock the other is waiting for. An earlier version locked `booking` before `show_seat` in the webhook handler and the sweeper. The load test caught 2 deadlocks in that version, and 0 after the order was fixed.

4. Idempotent payment webhook (`sp_process_payment_webhook`)

1. `SELECT ... FROM payment WHERE gateway=? AND gateway_order_id=? FOR UPDATE` locks the payment row, so concurrent deliveries for the same payment run one at a time.
2. `INSERT INTO payment_webhook_event` with `UNIQUE (gateway, gateway_event_id)`. A duplicate delivery hits error 1062, and the procedure returns `DUPLICATE_EVENT` without doing anything. The HTTP handler should still return 200 so the gateway stops retrying.
3. State changes only happen from `payment.status = 'INITIATED'`. A late `payment.failed` arriving after `payment.captured` is recorded but ignored (`ALREADY_SETTLED`).
4. On `payment.captured`, the procedure locks the booking's seats (same `seat_id` order), then the booking row, and confirms the booking **only if every seat still points at this booking**. If the hold expired and someone else took a seat, the booking becomes `FAILED` and the payment becomes `REFUND_PENDING`. The seats are never double sold.

5. Redis layer (fast path in front of MySQL)

At BookMyShow scale, most hold attempts for a hot show should be rejected before they ever reach MySQL:

- `SET seat:{show_id}:{seat_id} {booking_token} NX PX 600000` for each seat, run atomically across all requested seats with a Lua script (all-or-nothing). If any key already exists, the request is rejected immediately.
- If Redis grants the seats, the application calls `sp_hold_seats`. MySQL stays the **source of truth**: if Redis loses a key (failover, eviction), the conditional update still rejects a conflicting claim.
- The Redis TTL matches `hold_expires_at`, so the seat map (served from Redis) frees seats on the same timer.
- Burst handling: for launch-day spikes, hold requests go into a per-show queue (Redis Streams, SQS or Kafka partitioned by `show_id`). A small pool of workers drains it, which caps MySQL concurrency per show while users wait in a virtual queue.

Verified behaviour (MySQL 8.0, Docker)

Scenario	Result
User 1 holds A2+B1	HELD
User 2 requests A1+A2 (A2 held by user 1)	SEATS_UNAVAILABLE, and A1 is not held either (all-or-nothing)
User 1 retries with the same idempotency key	DUPLICATE_REQUEST, same booking_id
Success webhook delivered twice	CONFIRMED, then DUPLICATE_EVENT
Abandoned hold past its expiry	swept to EXPIRED by the event, seat freed
<code>tests/race_same_seats.sh 50</code> : 50 concurrent sessions, same 2 seats	1 × HELD, 49 × SEATS_UNAVAILABLE, 1 distinct owner, no deadlocks

Load test (designed and run)

`tests/load_test.py` builds an isolated 500-seat screen (rows A to T, 25 seats each) with one show, creates 2,000 users, and runs them through 100 concurrent database connections as a single burst. It models a hot show going on sale.

Workload design

Behaviour	Share	What it tests
Ask for 1 to 4 adjacent seats in the centre block (rows H to M, seats 8 to 18)	70% of attempts	Heavy contention on the same rows
Ask for seats anywhere in the hall	30% of attempts	Normal traffic
Pick other seats and try again if refused	up to 3 attempts	Retry storms
Re-send the identical hold request	10%	Idempotency key
Pay and get a <code>payment.captured</code> webhook	65% of holds	Confirmation path
Pay and get a <code>payment.failed</code> webhook	10% of holds	Release path
Abandon the cart	20% of holds	Hold expiry (8-second holds in the test)
Pay after the hold has expired	5% of holds	Late payment and refund path
Gateway sends the success webhook twice, at the same time	30% of captures	Webhook idempotency under concurrency
Background readers run the P2 listing query throughout	5 threads	Read latency under write load

Invariants checked after every run

1. No seat belongs to more than one `CONFIRMED` booking (no double booking).
2. Every `CONFIRMED` booking still owns all of its seats.
3. Every `CONFIRMED` booking has exactly one `SUCCESS` payment, for the right amount.
4. No seat is owned by a `FAILED` or `CANCELLED` booking.
5. No `SUCCESS` payment is attached to a booking that isn't `CONFIRMED`.
6. The number of `CONFIRMED` webhook responses equals the number of `CONFIRMED` bookings (nothing confirmed twice).
7. No lost holds: every payment that arrived before its hold expired was `CONFIRMED`.
8. A retried hold request always returned the original booking.
9. Once all holds have expired and the sweeper has run, no seat is still held.
10. Sold seats plus free seats equals total seats.

Results (MySQL 8.0 in Docker, laptop, 2,000 users, 100 connections)

duration: 25.1s virtual users: 2000 deadlock/lock-timeout retries: 0 unexpected errors: 0

operation	count	ops/s	p50 ms	p95 ms	p99 ms	max ms
hold	5611	223.4	57.6	174.1	234.8	332.2

hold_retry	530	21.1	54.3	158.8	221.1	309.5
webhook	264	10.5	80.1	194.9	226.1	298.7
webhook_dup	72	2.9	73.5	198.8	239.2	240.4
p2_listing	17300	688.8	2.5	29.6	111.3	314.7

hold:HELD 315 hold:SEATS_UNAVAILABLE 5296

webhook:CONFIRMED 207 (+13 from duplicate deliveries that won the race)

webhook:DUPLICATE_EVENT 13 (+59 duplicates rejected)

webhook:PAYMENT_FAILED 33 webhook:REFUND_REQUIRED 11 (late payments whose seats were re-sold)

final bookings: CONFIRMED 220, FAILED 44, EXPIRED 51

seats sold: 406/500

[PASS] all 10 invariants -> RESULT: ALL INVARIANTS HOLD

What the numbers show: - About 94% of hold attempts were refused. That is expected: 2,000 users were competing for 500 seats, mostly the same 66 centre seats. The refusals are fast (p50 of about 58 ms), and none of them caused a double booking. - 94 seats were left unsold. By the end, the free seats were either scattered gaps too small for the groups still asking, or seats released by failed and abandoned holds after most users had already given up (3 attempts each). - 11 late payments found their seats already re-sold. They were marked `REFUND_PENDING` rather than confirmed, as intended. - The P2 listing query stayed at about 2.5 ms median while the hold traffic ran, thanks to the `(screen_id, start_time)` index. - A second run gave the same result: all invariants passed, with 0 deadlocks.

P2: Shows at a theatre on a date, with timings

This is grouped the way the app displays it: one row per movie, language and format, with all its timings.

```
SET @theatre_id := 1;
SET @show_date  := '2026-09-30';

SELECT
  m.movie_id,
  m.title,
  m.certification,
  l.name AS language,
  f.name AS format,
  GROUP_CONCAT(DATE_FORMAT(ms.start_time, '%h:%i %p')
                ORDER BY ms.start_time SEPARATOR ', ') AS show_timings,
  COUNT(*) AS show_count
FROM screen      sc
JOIN movie_show  ms ON ms.screen_id = sc.screen_id
JOIN movie       m  ON m.movie_id   = ms.movie_id
JOIN language    l  ON l.language_id = ms.language_id
JOIN show_format f  ON f.format_id   = ms.format_id
WHERE sc.theatre_id = @theatre_id
      AND ms.start_time >= @show_date
      AND ms.start_time < @show_date + INTERVAL 1 DAY
```

```

AND ms.status = 'SCHEDULED'
GROUP BY m.movie_id, m.title, m.certification, l.name, f.name
ORDER BY m.title, l.name, f.name;

```

Output on the sample data (theatre 1 = PVR Orion Mall):

movie_id	title	certification	language	format	show_timings	show_count
2	Dune: Part Two	UA	English	IMAX 2D	05:00 PM	1
3	Inside Out 2	U	English	2D	12:30 PM	1
3	Inside Out 2	U	English	3D	10:00 AM	1
1	Kantara: Chapter 1	UA13+	Hindi	2D	07:00 PM	1
1	Kantara: Chapter 1	UA13+	Kannada	2D	09:30 AM, 01:15 PM, 09:00 PM	3
4	Oppenheimer	UA16+	English	2D	03:00 PM	1

The cancelled 10:30 PM Hindi show of Kantara is correctly left out.

sql/03_p2_shows_by_theatre_and_date.sql also has: - a **flat version** with one row per show, including show_id, so the client can open the seat map for a timing; - the **date-strip query** listing the next 7 dates that have shows.

Indexing for P2

- The date filter is a **half-open range** on start_time, not DATE(start_time) = ?. Wrapping the column in a function would prevent the index from being used.
- The access path is screen via uq_screen_theatre_name (theatre_id, ...) (about 5 to 15 screens per theatre), then movie_show via uq_show_screen_start (screen_id, start_time) as a range scan. EXPLAIN confirms both keys are used ("Using index" / "Using index condition"). Each screen reads only the roughly 5 shows of that day, whatever the size of the table.
- theatre_id was deliberately not copied onto movie_show. That would save one indexed join but break 3NF. If profiling ever showed the join to be the bottleneck, the next step would be a cache (the listing changes only when the schedule changes), not denormalizing the source of truth.

Appendix A: Full P2 script (sql/03_p2_shows_by_theatre_and_date.sql)

```

-- =====
-- P2: All shows at a given theatre on a given date, with show timings.
-- =====

USE booking_engine;

SET @theatre_id := 1;
SET @show_date := CURDATE(); -- e.g. '2026-09-30'

-- -----
-- P2 (a) - One row per movie/language/format with all its timings,

```

```
-- matching the reference UI ("Kantara: Chapter 1 | UA13+ | Kannada 2D |
-- 09:30 AM, 01:15 PM, 09:00 PM").
--
-- The date filter is a half-open range on start_time instead of
-- DATE(start_time) = @show_date, so it can use uq_show_screen_start
-- (screen_id, start_time) as a range scan.
```

```
-- -----
SELECT
    m.movie_id,
    m.title,
    m.certification,
    l.name AS language,
    f.name AS format,
    GROUP_CONCAT(DATE_FORMAT(ms.start_time, '%h:%i %p')
                  ORDER BY ms.start_time SEPARATOR ', ') AS show_timings,
    COUNT(*) AS show_count
FROM screen      sc
JOIN movie_show  ms ON ms.screen_id = sc.screen_id
JOIN movie       m  ON m.movie_id   = ms.movie_id
JOIN language    l  ON l.language_id = ms.language_id
JOIN show_format f  ON f.format_id   = ms.format_id
WHERE sc.theatre_id = @theatre_id
      AND ms.start_time >= @show_date
      AND ms.start_time < @show_date + INTERVAL 1 DAY
      AND ms.status = 'SCHEDULED'
GROUP BY m.movie_id, m.title, m.certification, l.name, f.name
ORDER BY m.title, l.name, f.name;
```

```
-- -----
-- P2 (b) - Flat version: one row per show (what an API would return so
-- the client can link each timing to its show_id for seat selection).
```

```
-- -----
SELECT
    ms.show_id,
    m.title,
    m.certification,
    l.name AS language,
    f.name AS format,
    sc.name AS screen,
    TIME_FORMAT(ms.start_time, '%h:%i %p') AS start_time,
    TIME_FORMAT(ms.end_time, '%h:%i %p') AS end_time
FROM screen      sc
JOIN movie_show  ms ON ms.screen_id = sc.screen_id
JOIN movie       m  ON m.movie_id   = ms.movie_id
JOIN language    l  ON l.language_id = ms.language_id
JOIN show_format f  ON f.format_id   = ms.format_id
WHERE sc.theatre_id = @theatre_id
      AND ms.start_time >= @show_date
      AND ms.start_time < @show_date + INTERVAL 1 DAY
```

```

    AND ms.status = 'SCHEDULED'
ORDER BY m.title, ms.start_time;

-- -----
-- Bonus: the date strip (next 7 dates that have at least one show).
-- -----

SELECT DISTINCT DATE(ms.start_time) AS show_date
FROM screen      sc
JOIN movie_show ms ON ms.screen_id = sc.screen_id
WHERE sc.theatre_id = @theatre_id
    AND ms.start_time >= CURDATE()
    AND ms.start_time <  CURDATE() + INTERVAL 7 DAY
    AND ms.status = 'SCHEDULED'
ORDER BY show_date;

-- Index check: expect `sc` via uq_screen_theatre_name and `ms` as a range
-- scan on uq_show_screen_start.
EXPLAIN
SELECT ms.show_id
FROM screen      sc
JOIN movie_show ms ON ms.screen_id = sc.screen_id
WHERE sc.theatre_id = @theatre_id
    AND ms.start_time >= @show_date
    AND ms.start_time <  @show_date + INTERVAL 1 DAY
    AND ms.status = 'SCHEDULED';

```

Appendix B: Booking flow stored procedures (`sql/04_booking_flow.sql`)

Seat hold, idempotent payment webhook, hold expiry job, and a demo run against the sample data. Run after the schema and sample data scripts.

```

-- =====
-- Concurrency-safe booking flow (supports P1's locking requirements)
--   sp_hold_seats           - all-or-nothing timed hold on N seats
--   sp_process_payment_webhook - idempotent webhook handler
--   sp_expire_holds         - background cleanup of expired holds
-- Run with the mysql CLI or MySQL Workbench (uses DELIMITER).
-- =====

USE booking_engine;

DROP PROCEDURE IF EXISTS sp_hold_seats;
DROP PROCEDURE IF EXISTS sp_process_payment_webhook;
DROP PROCEDURE IF EXISTS sp_expire_holds;
DROP EVENT IF EXISTS ev_expire_holds;

DELIMITER $$

```

```

-- -----
-- Hold seats for a user. A seat is claimable when nobody owns it, or its
-- owner booking is dead (EXPIRED/CANCELLED/FAILED), or its owner is a
-- PENDING hold whose timer has run out. Expiry is therefore enforced at
-- claim time and never depends on the cleanup job having run.
-- -----

CREATE PROCEDURE sp_hold_seats(
    IN p_user_id          BIGINT UNSIGNED,
    IN p_show_id          BIGINT UNSIGNED,
    IN p_seat_ids         JSON,          -- e.g. '[12, 13]'
    IN p_idempotency_key  CHAR(36),
    IN p_hold_seconds     INT,          -- typically 600 (10 minutes)
    OUT p_booking_id      BIGINT UNSIGNED,
    OUT p_result          VARCHAR(30)
)
proc: BEGIN
    DECLARE v_requested INT;
    DECLARE v_claimed  INT;
    DECLARE v_valid    INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET p_booking_id = NULL;

    -- Client retry with the same idempotency key: return the original booking.
    SELECT booking_id INTO p_booking_id
    FROM booking
    WHERE user_id = p_user_id AND idempotency_key = p_idempotency_key;
    IF p_booking_id IS NOT NULL THEN
        SET p_result = 'DUPLICATE_REQUEST';
        LEAVE proc;
    END IF;

    SELECT COUNT(DISTINCT j.seat_id) INTO v_requested
    FROM JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id INT UNSIGNED PATH '$')) j;

    IF v_requested = 0 OR v_requested <> JSON_LENGTH(p_seat_ids) THEN
        SET p_result = 'INVALID_SEATS';
        LEAVE proc;
    END IF;

    START TRANSACTION;

    -- Show must be bookable, and every seat must belong to it.
    SELECT COUNT(*) INTO v_valid

```

```

FROM movie_show ms
JOIN show_seat ss ON ss.show_id = ms.show_id
JOIN JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id INT UNSIGNED PATH '$')) j
    ON j.seat_id = ss.seat_id
WHERE ms.show_id = p_show_id
    AND ms.status = 'SCHEDULED'
    AND ms.start_time > NOW();

IF v_valid <> v_requested THEN
    ROLLBACK;
    SET p_result = 'INVALID_SEATS';
    LEAVE proc;
END IF;

INSERT INTO booking (user_id, show_id, status, hold_expires_at, idempotency_key)
VALUES (p_user_id, p_show_id, 'PENDING', NOW() + INTERVAL p_hold_seconds SECOND, p_idempotency_key);
SET p_booking_id = LAST_INSERT_ID();

-- Pessimistic lock on the requested inventory rows, always in seat_id
-- order so two overlapping requests cannot deadlock each other.
SELECT COUNT(*) INTO v_valid FROM (
    SELECT ss.seat_id
    FROM show_seat ss
    JOIN JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id INT UNSIGNED PATH '$')) j
        ON j.seat_id = ss.seat_id
    WHERE ss.show_id = p_show_id
    ORDER BY ss.seat_id
    FOR UPDATE OF ss
) locked;

-- Claim only the seats that are actually free right now.
UPDATE show_seat ss
JOIN JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id INT UNSIGNED PATH '$')) j
    ON j.seat_id = ss.seat_id
LEFT JOIN booking b ON b.booking_id = ss.booking_id
SET ss.booking_id = p_booking_id,
    ss.version = ss.version + 1
WHERE ss.show_id = p_show_id
    AND ( ss.booking_id IS NULL
        OR b.status IN ('EXPIRED', 'CANCELLED', 'FAILED')
        OR (b.status = 'PENDING' AND b.hold_expires_at <= NOW()));
SET v_claimed = ROW_COUNT();

-- All-or-nothing: if any seat was taken, undo everything.
IF v_claimed <> v_requested THEN
    ROLLBACK;
    SET p_booking_id = NULL;
    SET p_result = 'SEATS_UNAVAILABLE';
    LEAVE proc;

```

```

END IF;

INSERT INTO booking_seat (booking_id, seat_id, price_paid)
SELECT p_booking_id, s.seat_id, sp.price
FROM JSON_TABLE(p_seat_ids, '$[*]' COLUMNS (seat_id INT UNSIGNED PATH '$')) j
JOIN seat s          ON s.seat_id = j.seat_id
JOIN show_price sp ON sp.show_id = p_show_id AND sp.category_id = s.category_id;

COMMIT;

SET p_result = 'HELD';
END proc$$

-- -----
-- Payment webhook. Safe to call any number of times for the same event:
-- * UNIQUE (gateway, gateway_event_id) rejects repeat deliveries;
-- * the payment row is locked FOR UPDATE, serialising concurrent
--   deliveries for the same payment;
-- * state transitions only happen from INITIATED / PENDING.
-- The booking is confirmed only if it still owns every one of its seats.
-- A late payment whose seats were already re-sold is marked for refund.
-- -----

CREATE PROCEDURE sp_process_payment_webhook(
    IN  p_gateway          VARCHAR(30),
    IN  p_gateway_event_id VARCHAR(100),
    IN  p_gateway_order_id VARCHAR(100),
    IN  p_event_type        VARCHAR(50),
    IN  p_payload           JSON,
    OUT p_result            VARCHAR(30)
)
proc: BEGIN
    DECLARE v_payment_id    BIGINT UNSIGNED;
    DECLARE v_booking_id    BIGINT UNSIGNED;
    DECLARE v_payment_status VARCHAR(20);
    DECLARE v_booking_status VARCHAR(20);
    DECLARE v_total_seats   INT;
    DECLARE v_owned_seats   INT;
    DECLARE v_show_id       BIGINT UNSIGNED;
    DECLARE v_duplicate     TINYINT DEFAULT 0;

    DECLARE CONTINUE HANDLER FOR 1062 SET v_duplicate = 1;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT payment_id, booking_id, status

```

```

    INTO v_payment_id, v_booking_id, v_payment_status
FROM payment
WHERE gateway = p_gateway AND gateway_order_id = p_gateway_order_id
FOR UPDATE;

IF v_payment_id IS NULL THEN
    ROLLBACK;
    SET p_result = 'UNKNOWN_PAYMENT';
    LEAVE proc;
END IF;

INSERT INTO payment_webhook_event (gateway, gateway_event_id, payment_id, event_type, payload)
VALUES (p_gateway, p_gateway_event_id, v_payment_id, p_event_type, p_payload);

IF v_duplicate = 1 THEN
    ROLLBACK;
    SET p_result = 'DUPLICATE_EVENT';
    LEAVE proc;
END IF;

IF v_payment_status <> 'INITIATED' THEN
    -- Different event id but payment already settled (e.g. a late
    -- payment.failed after payment.captured). Record, don't act.
    UPDATE payment_webhook_event SET processed_at = NOW()
    WHERE gateway = p_gateway AND gateway_event_id = p_gateway_event_id;
    COMMIT;
    SET p_result = 'ALREADY_SETTLED';
    LEAVE proc;
END IF;

-- Lock order everywhere is show_seat rows (by seat_id) before booking
-- rows, matching sp_hold_seats, so the two can never deadlock.
SELECT show_id INTO v_show_id FROM booking WHERE booking_id = v_booking_id;

SELECT COUNT(*), SUM(ss.booking_id <=> v_booking_id)
    INTO v_total_seats, v_owned_seats
FROM (
    SELECT ss.booking_id
    FROM booking_seat bs
    JOIN show_seat ss ON ss.show_id = v_show_id AND ss.seat_id = bs.seat_id
    WHERE bs.booking_id = v_booking_id
    ORDER BY ss.seat_id
    FOR UPDATE OF ss
) ss;

SELECT status INTO v_booking_status
FROM booking WHERE booking_id = v_booking_id
FOR UPDATE;

```

```

IF p_event_type = 'payment.captured' THEN
    IF v_booking_status = 'PENDING' AND v_total_seats > 0 AND v_owned_seats = v_total_seats THEN
        UPDATE booking SET status = 'CONFIRMED' WHERE booking_id = v_booking_id;
        UPDATE payment SET status = 'SUCCESS' WHERE payment_id = v_payment_id;
        SET p_result = 'CONFIRMED';
    ELSE
        UPDATE booking SET status = 'FAILED' WHERE booking_id = v_booking_id AND status =
'PENDING';
        UPDATE show_seat SET booking_id = NULL, version = version + 1
        WHERE show_id = v_show_id AND booking_id = v_booking_id;
        UPDATE payment SET status = 'REFUND_PENDING' WHERE payment_id = v_payment_id;
        SET p_result = 'REFUND_REQUIRED';
    END IF;
ELSEIF p_event_type = 'payment.failed' THEN
    UPDATE booking SET status = 'FAILED' WHERE booking_id = v_booking_id AND status = 'PENDING';
    UPDATE show_seat SET booking_id = NULL, version = version + 1
    WHERE show_id = v_show_id AND booking_id = v_booking_id;
    UPDATE payment SET status = 'FAILED' WHERE payment_id = v_payment_id;
    SET p_result = 'PAYMENT_FAILED';
ELSE
    SET p_result = 'IGNORED_EVENT_TYPE';
END IF;

UPDATE payment_webhook_event SET processed_at = NOW()
WHERE gateway = p_gateway AND gateway_event_id = p_gateway_event_id;

COMMIT;
END proc$$

```

```

-- -----
-- Housekeeping: mark expired holds and detach them from inventory in
-- small batches (keeps lock time short). Correctness does not depend on
-- this running - sp_hold_seats already treats expired holds as free.
-- Skips holds whose payment is still INITIATED within a grace window,
-- so an in-flight payment is not raced by the sweeper.
-- -----

```

```

CREATE PROCEDURE sp_expire_holds(IN p_batch_size INT, OUT p_expired INT)
BEGIN
    DECLARE v_locked INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    DROP TEMPORARY TABLE IF EXISTS tmp_expired;
    CREATE TEMPORARY TABLE tmp_expired (booking_id BIGINT UNSIGNED PRIMARY KEY);

```

```

-- READ COMMITTED: candidate selection takes no locks on booking, so
-- seats can be locked before bookings (same order as sp_hold_seats).
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

INSERT INTO tmp_expired (booking_id)
SELECT b.booking_id
FROM booking b
WHERE b.status = 'PENDING'
      AND b.hold_expires_at <= NOW()
      AND NOT EXISTS (
        SELECT 1 FROM payment p
        WHERE p.booking_id = b.booking_id
              AND p.status = 'INITIATED'
              AND b.hold_expires_at > NOW() - INTERVAL 5 MINUTE
      )
ORDER BY b.hold_expires_at
LIMIT p_batch_size;

SELECT COUNT(*) INTO v_locked FROM (
  SELECT ss.seat_id
  FROM show_seat ss
  JOIN tmp_expired t ON t.booking_id = ss.booking_id
  ORDER BY ss.show_id, ss.seat_id
  FOR UPDATE OF ss
) locked;

-- Re-check: a webhook may have confirmed the booking meanwhile.
UPDATE booking b JOIN tmp_expired t ON t.booking_id = b.booking_id
SET b.status = 'EXPIRED'
WHERE b.status = 'PENDING' AND b.hold_expires_at <= NOW();
SET p_expired = ROW_COUNT();

UPDATE show_seat ss
JOIN tmp_expired t ON t.booking_id = ss.booking_id
JOIN booking b    ON b.booking_id = ss.booking_id
SET ss.booking_id = NULL, ss.version = ss.version + 1
WHERE b.status = 'EXPIRED';

COMMIT;
DROP TEMPORARY TABLE IF EXISTS tmp_expired;
END$$

DELIMITER ;

-- Runs every 30s if event_scheduler=ON (default in MySQL 8).
CREATE EVENT ev_expire_holds
ON SCHEDULE EVERY 30 SECOND
DO CALL sp_expire_holds(1000, @ev_expired);

```

```

-- =====
-- Demo against the sample data (today's 09:30 show at PVR Orion, Audi 1)
-- =====

SET @show_id := (SELECT show_id FROM movie_show
                  WHERE screen_id = 1 AND start_time = TIMESTAMP(CURDATE(), '09:30:00'));
-- If it's already past 09:30 today, use tomorrow's 09:30 show instead
-- (sp_hold_seats refuses shows that have started).
SET @show_id := IF((SELECT start_time FROM movie_show WHERE show_id = @show_id) > NOW(), @show_id,
                  (SELECT show_id FROM movie_show
                   WHERE screen_id = 1 AND start_time = TIMESTAMP(CURDATE() + INTERVAL 1 DAY,
                           '09:30:00')));

SET @a1 := (SELECT seat_id FROM seat WHERE screen_id = 1 AND row_label = 'A' AND seat_number = 1);
SET @a2 := (SELECT seat_id FROM seat WHERE screen_id = 1 AND row_label = 'A' AND seat_number = 2);
SET @b1 := (SELECT seat_id FROM seat WHERE screen_id = 1 AND row_label = 'B' AND seat_number = 1);

-- 1. User 1 holds A2 + B1 -> HELD
CALL sp_hold_seats(1, @show_id, JSON_ARRAY(@a2, @b1), '11111111-0000-4000-8000-000000000001', 600, @bk1,
                  @r1);
-- 2. User 2 tries A2 (overlaps user 1's live hold) -> SEATS_UNAVAILABLE, nothing held
CALL sp_hold_seats(2, @show_id, JSON_ARRAY(@a1, @a2), '22222222-0000-4000-8000-000000000002', 600, @bk2,
                  @r2);
-- 3. User 1 retries request #1 (network retry) -> DUPLICATE_REQUEST, same booking id
CALL sp_hold_seats(1, @show_id, JSON_ARRAY(@a2, @b1), '11111111-0000-4000-8000-000000000001', 600, @bk3,
                  @r3);
SELECT @r1, @bk1, @r2, @bk2, @r3, @bk3;

-- 4. Payment for user 1's hold; gateway delivers the success webhook twice.
INSERT INTO payment (booking_id, gateway, gateway_order_id, amount)
SELECT @bk1, 'RAZORPAY', CONCAT('order_demo_', @bk1), SUM(price_paid)
FROM booking_seat WHERE booking_id = @bk1;

CALL sp_process_payment_webhook('RAZORPAY', CONCAT('evt_demo_', @bk1), CONCAT('order_demo_', @bk1),
                               'payment.captured', JSON_OBJECT('status', 'captured'), @w1);
CALL sp_process_payment_webhook('RAZORPAY', CONCAT('evt_demo_', @bk1), CONCAT('order_demo_', @bk1),
                               'payment.captured', JSON_OBJECT('status', 'captured'), @w2);
SELECT @w1 AS first_delivery, @w2 AS redelivery; -- CONFIRMED, DUPLICATE_EVENT

-- 5. Run the sweeper manually.
CALL sp_expire_holds(1000, @expired);
SELECT @expired AS holds_expired;

-- Seat map for the show: AVAILABLE / HELD / BOOKED is derived, not stored.
SELECT s.row_label, s.seat_number, sc.name AS category,
CASE
    WHEN b.status = 'CONFIRMED' THEN 'BOOKED'
    WHEN b.status = 'PENDING' AND b.hold_expires_at > NOW() THEN 'HELD'

```

```
        ELSE 'AVAILABLE'
    END AS seat_state
FROM show_seat ss
JOIN seat s      ON s.seat_id = ss.seat_id
JOIN seat_category sc ON sc.category_id = s.category_id
LEFT JOIN booking b  ON b.booking_id = ss.booking_id
WHERE ss.show_id = @show_id
ORDER BY s.row_label, s.seat_number;
```